RTOS GPIO

开发指南

版本号: 1.5

发布日期: 2026.02.09

深圳易新泰科技有限公司YiXinTai

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.07.09	ALLWINNER	1. 初始版本
1.1	2023.02.14	XAA0249	1. 修改格式
1.2	2024.10.22	AWA1427	1. 基于新模板更新文档
1.3	2025.07.01	AWA2298	1. 增加平台描述
1.4	2025.11.19	XAA0338	1. 更新部分章节的标题 2. 更新函数接口描述 3. 增加 FAQ 章节
1.5	2026.02.09	XAA0338	1. 更新适用产品列表

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

目录

第 1 章 前言 1
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 1
1.4.1 标志说明 1
1.4.2 地址与数据描述方法约定 2
1.4.3 数值单位约定 2
1.5 相关术语介绍 2
1.5.1 硬件术语 2
1.5.2 软件术语 2

第 2 章 模块介绍 3
2.1 模块功能 3
2.2 模块配置 3
2.2.1 sys_config.fex 配置说明 3
2.2.2 menuconfig 配置说明 4
2.2.3 gpio_irq_map 配置说明 4
2.3 源码结构 4
2.4 软件框架 5

第 3 章 模块接口说明 6
3.1 数据结构 6
3.1.1 gpio_pin_t 6
3.1.2 gpio_driving_level_t 6
3.1.3 gpio_pull_status_t 7
3.1.4 gpio_data_t 7
3.1.5 gpio_power_mode_t 7
3.1.6 gpio_interrupt_mode_t 7
3.2 函数接口 7
3.2.1 hal_gpio_check_valid 7
3.2.2 hal_gpio_get_data 8
3.2.3 hal_gpio_set_data 8
3.2.4 hal_gpio_set_direction 8
3.2.5 hal_gpio_get_direction 9
3.2.6 hal_gpio_set_pull 9
3.2.7 hal_gpio_get_pull 10
3.2.8 hal_gpio_set_driving_level 10
3.2.9 hal_gpio_get_driving_level 10
3.2.10 hal_gpio_pinmux_set_function 11
3.2.11 hal_gpio_pinmux_get_function 11
3.2.12 hal_gpio_sel_vol_mode 11
3.2.13 hal_gpio_set_debounce 12
3.2.14 hal_gpio_to_irq 12
3.2.15 hal_gpio_irq_request 12
3.2.16 hal_gpio_irq_free 13
3.2.17 hal_gpio_irq_enable 13
3.2.18 hal_gpio_irq_disable 13
3.2.19 hal_gpio_init 14

第 4 章 功能开发 15
4.1 功能概述 15
4.2 开发流程 15
4.2.1 配置 GPIO 为输入模式并读取状态 15

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

4.2.2 配置 GPIO 为输出模式并设置输出电平 . . . 15
4.2.3 配置 GPIO 为其它模块功能引脚 . . . 15
4.2.4 配置 GPIO 中断模式 . . . 16
4.3 编程示例 . . . 16

第 5 章 调试方法 18
5.1 调试信息 . . . 18
5.2 调试工具 . . . 18
5.2.1 hal_gpio . . . 18
5.2.2 hal_gpio_output . . . 18
5.2.3 hal_gpio_input . . . 18
5.2.4 hal_gpio_irq . . . 19
5.2.5 hal_gpio_info . . . 19

第 6 章 FAQ 20
6.1 常见问题及解决方法 . . . 20
6.1.1 GPIO 中断无法触发 . . . 20
6.1.2 GPIO 电平读取值异常 . . . 20
6.1.3 GPIO 输出电平无法改变 . . . 20
6.1.4 系统休眠唤醒后 GPIO 配置丢失 . . . 21
6.2 一号通 FAQ 列表 . . . 21

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

表格

表 1-1 适用产品列表 1
表 1-2 地址与数据描述方法约定 2
表 1-3 数值单位约定 2
表 1-4 硬件术语 2
表 1-5 软件术语 2
表 3-1 模块接口列表 6

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 1 章 前言

1.1 文档简介

本文档介绍 Allwinner 平台 RTOS 中 GPIO 驱动的接口及使用方法。

1.2 目标读者

- RTOS GPIO 模块开发工程师
- RTOS GPIO 模块使用工程师

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
V459	Melis	hal_gpio.c
F133	Melis	hal_gpio.c
F135	Melis	hal_gpio.c
R328	FreeRTOS	hal_gpio.c
R329-DSP	FreeRTOS	hal_gpio.c
R128	FreeRTOS	hal_gpio.c
XR875	FreeRTOS	hal_gpio.c
MR527	FreeRTOS	hal_gpio.c
V821	FreeRTOS	hal_gpio.c
V861	FreeRTOS	hal_gpio.c
V881	FreeRTOS	hal_gpio.c
V838	FreeRTOS	hal_gpio.c
F102	Melis	hal_gpio.c
F101	FreeRTOS	hal_gpio.c

1.4 文档约定

1.4.1 标志说明

注意

- 提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示（寄存器描述除外）。
X	00X，XX1	数据描述中，X 代表 0 或 1。（例如，00X 代表 000 或 001；XX1 代表 001、011、101 或 111。）

1.4.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

表 1-3: 数值单位约定

类型	符号	对应数值
数据容量 (如 NAND 容量)	1 K 1 M 1 G	1024 1 048 576 1 073 741 824
频率，数据速率等	1 K 1 M 1 G	1000 1 000 000 1 000 000 000

1.5 相关术语介绍

1.5.1 硬件术语

表 1-4: 硬件术语

术语	解释说明
MUX	multiplexer，数据选择器

1.5.2 软件术语

表 1-5: 软件术语

术语	解释说明
GPIO	General Purpose Input/Output，通用输入输出
HAL	Hardware Abstraction Layer，硬件抽象层

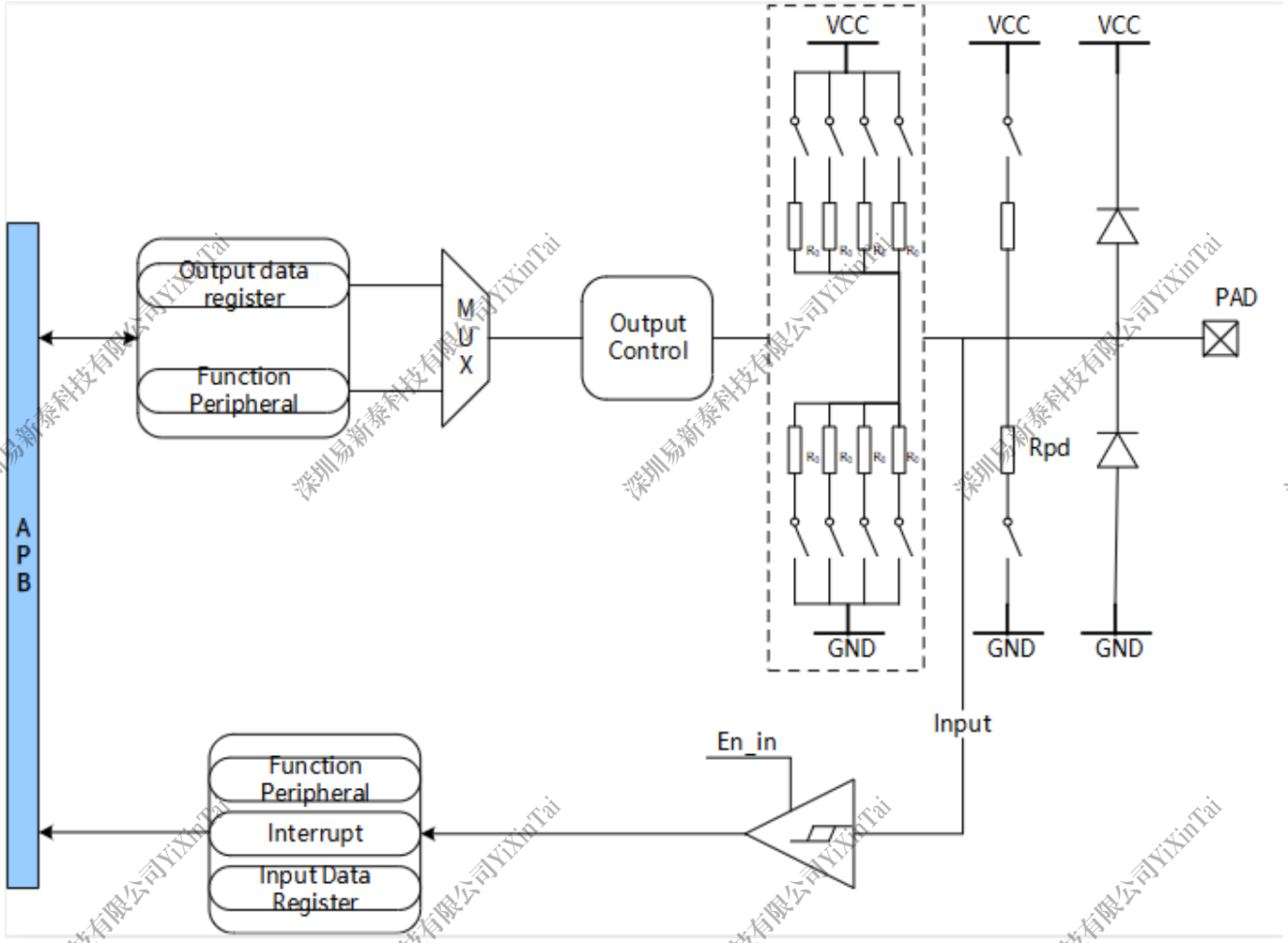
深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 2 章 模块介绍

2.1 模块功能

图 2-1: GPIO 控制器组成示意图



整个 GPIO 控制器由数字部分 (GPIO 和外设接口) 以及 IO 模拟部分 (输出缓冲, 双下拉,pad) 组成, 如图 2-1 所示。其中数字部分的输出可以通过 MUX 开关选择, 模拟部分可以用来配置上下拉, 驱动能力以及引脚输出电压等等。具体的规格如下:

- 可以在软件上配置各个引脚的状态
- 每个引脚都可以触发中断
- 可以配置上拉/下拉/无上下拉三种状态
- 每个引脚都可以配置 4 种驱动能力
- 支持中断的引脚可以配置为边沿触发或水平触发
- 中断数量跟随平台变化

2.2 模块配置

2.2.1 sys_config.fex 配置说明

其它模块若需要使用 GPIO，需要在 sys_config 中配置。

在描述 GPIO 的形式如下:

版权所有 © 珠海全志科技股份有限公司。保留一切权利

Port: 端口 + 组内序号

示例如下

```
uart_tx    = port:PA8<5><1><default><default>
```

2.2.2 menuconfig 配置说明

```
Drivers Options  --->
    soc related device drivers  --->
         [*] enable gpio driver    #使能GPIO驱动
         [ ]   enable gpio hal APIs test command   #GPIO驱动测试代码
         [ ] enable gpio chip aw9523    #支持GPIO拓展芯片
         [ ] enable amp gpio irq   #适用多个核都跑RTOS的平台，如R128/XR875
         [*] enable record gpio usage for debug #记录GPIO的使用情况，V821后才开始添加
```

 说明

1. “enable amp gpio irq” 选项适用多个核都跑 RTOS 的平台（如 R128/XR875），对应 CONFIG_DRIVERS_GPIO_SHARE_IRQ 配置，避免同一 port 的 GPIO 在多个核触发中断，。GPIO 中断的分配需要在 sys_config.fex 的 “gpio_irq_map” 配置。
2. Linux + RTOS 的 AMP 平台是 CONFIG_AMP_SHARE_IRQ 配置，未在 GPIO 驱动中配置，并且中断分配情况需要在 Linux 端配置。

2.2.3 gpio_irq_map 配置说明

适用于 R128/XR875 平台等多个核都跑 RTOS 的平台。

（1）sys_config.fex 文件中 gpio_irq_map 主键，用于表示各个核使用 gpio 中断的情况；

（2）gpio_1_1、gpio_1_2 中每两个 bit 表示对应的 pin 的中断由哪个核负责处理。其中，0x0 表示不使用，0x1 表示 M33，0x2 表示 C906；

（3）比如 gpio_1_1 为 0xAAAAAAA，则表示 PA0~PA15 对应 gpio 中断由 c906 处理；

配置示例：

```
将PA19配置给M33使用（以便M33能使能其中断）

[gpio_irq_map]
gpio_1_1 = 0xAAAAAAAA
gpio_1_2 = 0xAAAAAA6A
gpio_2_1 = 0xAAAAAAAA
gpio_2_2 = 0xAAAAAAAA
gpio_3_1 = 0xAAAAAAAA
gpio_3_2 = 0xAAAAAAAA
```

2.3 源码结构

1. 驱动源码

```
hal/source/gpio/
├── aw9523.c
├── aw9523.h
├── gpio.h
├── hal_gpio.c
├── Kconfig
├── Makefile
├── sun20iw2
│   ├── gpio-sun20iw2.c
│   ├── Makefile
│   └── platform-gpio.h
├── sun300iw1
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

```
|   ├── gpio-sun300iw1.c
|   ├── Makefile
|   └── platform-gpio.h
├── sun50iw11
|   ├── gpio-sun50iw11.c
|   ├── Makefile
|   └── platform-gpio.h
├── sun55iw3
|   ├── gpio-sun55iw3.c
|   ├── Makefile
|   └── platform-gpio.h
....
....
```

2. 驱动 APIs 声明头文件

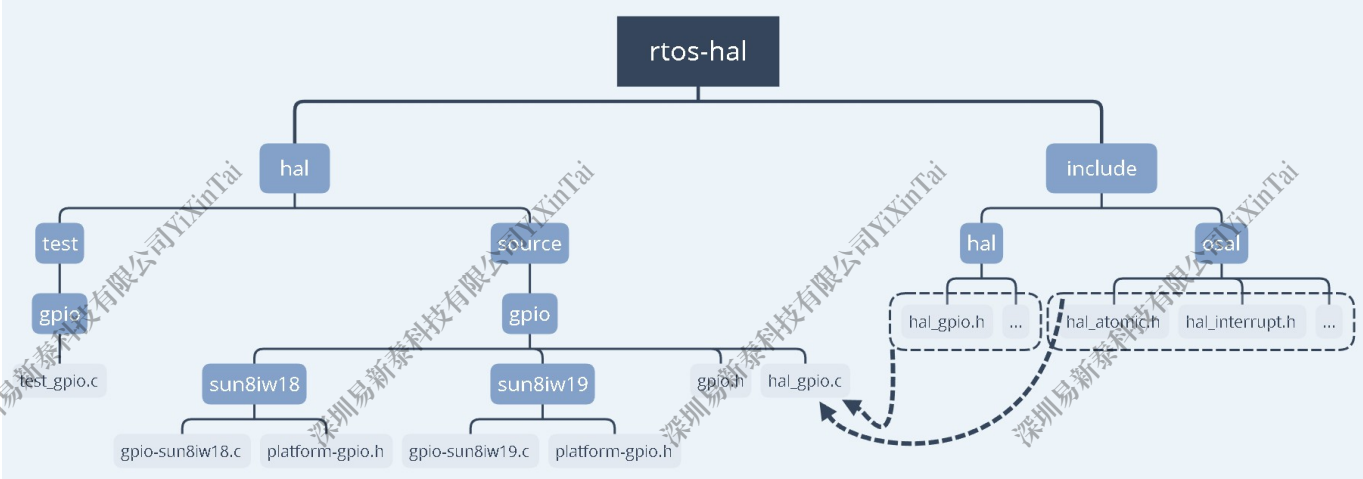
```
include/hal/
└── hal_gpio.h
```

3. 驱动 APIs 测试代码

```
hal/test/gpio/
├── Makefile
├── test_conf.h
└── test_gpio.c
```

2.4 软件框架

图 2-2: RTOS GPIO 驱动软件框架图



RTOS GPIO 驱动采用分层架构设计 (参见图 2-2), 从上到下分为应用层、HAL 抽象层和硬件驱动层, 为上层应用提供统一的 GPIO 操作接口。

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 3 章 模块接口说明

表 3-1: 模块接口列表

API	解释说明
hal_gpio_check_valid	判断 GPIO 的合法性
hal_gpio_get_data	获取指定 GPIO 的电平状态
hal_gpio_set_data	设置指定 GPIO 的电平状态
hal_gpio_set_direction	设置指定 GPIO 的 IO 模式
hal_gpio_get_direction	获取指定 GPIO 的 IO 模式
hal_gpio_set_pull	设置指定 GPIO 的上下拉状态
hal_gpio_get_pull	获取指定 GPIO 的上下拉状态
hal_gpio_set_driving_level	设置指定 GPIO 的驱动能力
hal_gpio_get_driving_level	获取指定 GPIO 的驱动能力
hal_gpio_pinmux_set_function	设置指定 GPIO 的复用功能
hal_gpio_pinmux_get_function	获取指定 GPIO 的复用功能
hal_gpio_sel_vol_mode	设置指定 GPIO 组的电压模式
hal_gpio_set_debounce	设置指定 GPIO 组的中断采样频率
hal_gpio_to_irq	获取指定 GPIO 的 IRQ 中断号
hal_gpio_irq_request	GPIO 中断申请
hal_gpio_irq_free	GPIO 中断释放
hal_gpio_irq_enable	使能 GPIO 中断
hal_gpio_irq_disable	关闭 GPIO 中断
hal_gpio_init	GPIO 驱动初始化

3.1 数据结构

由于 GPIO 需要配置每个引脚的引脚复用功能，中断类型，驱动能力，上下拉，输出/输入数据，输入/输出方向等等，所以对 GPIO 的这些配置都封装在一个 enum 枚举结构里面，方便使用。下面是一些配置的定义，想要了解更多的数据结构可以到 hal_gpio.h 查看。

3.1.1 gpio_pin_t

该枚举定义了可用的每个引脚定义，在配置引脚的时候将相关参数传入则可，具体定义如下：

```
typedef enum
{
    GPIO_PC0 = GPIOC(0),
    GPIO_PC1 = GPIOC(1),
    GPIO_PC2 = GPIOC(2),
    GPIO_PC3 = GPIOC(3),
    ...
    GPIO_PL0 = GPIOL(0),
    GPIO_PL1 = GPIOL(1),
    GPIO_PL2 = GPIOL(2),
    GPIO_PL3 = GPIOL(3),
    GPIO_PL4 = GPIOL(4),
    GPIO_PL5 = GPIOL(5),
} gpio_pin_t;
```

3.1.2 gpio_driving_level_t

该枚举定义了引脚的驱动能力的值，具体定义如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
typedef enum
{
    GPIO_DRIVING_LEVEL0    = 0,         /**< Defines GPIO driving current as level0.  */
    GPIO_DRIVING_LEVEL1    = 1,         /**< Defines GPIO driving current as level1.  */
    GPIO_DRIVING_LEVEL2    = 2,         /**< Defines GPIO driving current as level2. */
    GPIO_DRIVING_LEVEL3    = 3          /**< Defines GPIO driving current as level3. */
} gpio_driving_level_t;
```

3.1.3 gpio_pull_status_t

该枚举定义了引脚的上下拉的值，具体定义如下：

```
typedef enum
{
    GPIO_PULL_DOWN_DISABLED    = 0,        /**< Defines GPIO pull up and pull down disable.  */
    GPIO_PULL_UP               = 1,        /**< Defines GPIO is pull up state.  */
    GPIO_PULL_DOWN             = 2,        /**< Defines GPIO is pull down state. */
} gpio_pull_status_t;
```

3.1.4 gpio_data_t

该枚举定义引脚的输入输出数据，具体定义如下：

```
typedef enum
{
    GPIO_DATA_LOW  = 0,                    /**<  GPIO data low. */
    GPIO_DATA_HIGH = 1                     /**<  GPIO data high. */
} gpio_data_t;
```

3.1.5 gpio_power_mode_t

该枚举定义了引脚的电压模式，可以配置成 1.8V 和 3.3V，具体定义如下

```
typedef enum
{
    POWER_MODE_330 = 0,
    POWER_MODE_180 = 1
} gpio_power_mode_t;
```

3.1.6 gpio_interrupt_mode_t

该枚举定义了引脚的中断模式，具体定义如下：

```
typedef enum
{
    IRQ_TYPE_NONE       = 0x00000000,
    IRQ_TYPE_EDGE_RISING    = 0x00000001,
    IRQ_TYPE_EDGE_FALLING   = 0x00000002,
    IRQ_TYPE_EDGE_BOTH  = (IRQ_TYPE_EDGE_FALLING | IRQ_TYPE_EDGE_RISING),
    IRQ_TYPE_LEVEL_HIGH = 0x00000004,
    IRQ_TYPE_LEVEL_LOW  = 0x00000008,
} gpio_interrupt_mode_t;
```

3.2 函数接口

3.2.1 hal_gpio_check_valid

函数原型：

```
bool hal_gpio_check_valid(gpio_pin_t pin)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

作用：

判断 GPIO 的合法性。

参数：

pin：pin id。

返回：

true：合法。

false：不合法。

3.2.2 hal_gpio_get_data

函数原型：

```
int hal_gpio_get_data(gpio_pin_t pin, gpio_data_t *data)
```

作用：

获取指定 GPIO 的电平状态。

参数：

pin：pin id。

data：存放电平的指针变量。

返回：

-1：失败。

0：成功。

3.2.3 hal_gpio_set_data

函数原型：

```
int hal_gpio_set_data(gpio_pin_t pin, gpio_data_t data)
```

作用：

设置指定 GPIO 的电平状态。

参数：

pin：pin id。

data：需设置的电平高低。

返回：

-1：失败。

0：成功。

3.2.4 hal_gpio_set_direction

函数原型：

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
int hal_gpio_set_direction(gpio_pin_t pin, gpio_direction_t direction)
```

作用：

设置指定 GPIO 的 IO 模式。

参数：

pin：pin id。

direction：需设置的 IO 模式。

返回：

-1：失败。

0：成功。

3.2.5 hal_gpio_get_direction

函数原型：

```
int hal_gpio_get_direction(gpio_pin_t pin, gpio_direction_t *direction)
```

作用：

获取指定 GPIO 的 IO 模式。

参数：

pin：pin id。

direction：存放 IO 模式的指针变。

返回：

-1：失败。

0：成功。

3.2.6 hal_gpio_set_pull

函数原型：

```
int hal_gpio_set_pull(gpio_pin_t pin, gpio_pull_status_t pull)
```

作用：

设置指定 GPIO 的上下拉状态。

参数：

pin：pin id。

pull：需设置的上下拉状态。

返回：

-1：失败。

0：成功。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

3.2.7 hal_gpio_get_pull

函数原型:

```
int hal_gpio_get_pull(gpio_pin_t pin, gpio_pull_status_t *pull)
```

作用:

获取指定 GPIO 的上下拉状态。

参数:

pin：pin id。

pull：存放上下拉状态的变量。

返回:

-1：失败。

0：成功。

3.2.8 hal_gpio_set_driving_level

函数原型:

```
int hal_gpio_set_driving_level(gpio_pin_t pin, gpio_driving_level_t level)
```

作用:

设置指定 GPIO 的驱动能力。

参数:

pin：pin id。

level：需设置的驱动能力。

返回:

-1：失败。

0：成功。

3.2.9 hal_gpio_get_driving_level

函数原型:

```
int hal_gpio_get_driving_level(gpio_pin_t pin, gpio_driving_level_t *level)
```

作用:

获取指定 GPIO 的驱动能力。

参数:

pin：pin id。

level：存放驱动能力的变量。

返回:

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

-1：失败。

0：成功。

3.2.10 hal_gpio_pinmux_set_function

函数原型：

```
int hal_gpio_pinmux_set_function(gpio_pin_t pin, gpio_muxsel_t function_index)
```

作用：

设置指定 GPIO 的复用功能。

参数：

pin：pin id。

function_index：需设置的复用功能。

返回：

-1：失败。

0：成功。

3.2.11 hal_gpio_pinmux_get_function

函数原型：

```
int hal_gpio_pinmux_get_function(gpio_pin_t pin, gpio_muxsel_t *function_index)
```

作用：

获取指定 GPIO 的复用功能。

参数：

pin：pin id。

function_index：存放复用功能的指针变量。

返回：

-1：失败。

0：成功。

3.2.12 hal_gpio_sel_vol_mode

函数原型：

```
int hal_gpio_sel_vol_mode(gpio_pin_t pins, gpio_power_mode_t pm_sel)
```

作用：

设置指定 GPIO 组的电压模式。

参数：

pin：pin id。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

pm_sel：需设置的电压模式。

返回：

-1：失败。

0：成功。

3.2.13 hal_gpio_set_debounce

函数原型：

```
int hal_gpio_set_debounce(gpio_pin_t pin, unsigned value)
```

作用：

设置指定 GPIO 组的中断采样频率。

参数：

pin：pin id。

value：需设置的值（bit0-clock select，bit6:4-clock pre-scale）。

返回：

-1：失败。

0：成功。

3.2.14 hal_gpio_to_irq

函数原型：

```
int hal_gpio_to_irq(gpio_pin_t pin, uint32_t *irq)
```

作用：

获取指定 GPIO 的 IRQ 中断号。

参数：

pin：pin id。

irq：存放中断号的指针变量。

返回：

-1：失败。

0：成功。

3.2.15 hal_gpio_irq_request

函数原型：

```
int hal_gpio_irq_request(uint32_t irq, hal_irq_handler_t hdle, unsigned long flags, void *data)
```

作用：

GPIO 中断申请。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

参数:

irq：中断号。

hdle：中断处理函数。

flags：中断触发模式。

data：私有数据指针变量。

返回:

-1：失败。

0：成功。

3.2.16 hal_gpio_irq_free

函数原型:

```
int hal_gpio_irq_free(uint32_t irq)
```

作用:

GPIO 中断释放。

参数:

irq：中断号。

返回:

-1：失败。

irq：成功释放的 irq 号。

3.2.17 hal_gpio_irq_enable

函数原型:

```
int hal_gpio_irq_enable(uint32_t irq)
```

作用:

使能 GPIO 中断。

参数:

irq：中断号。

返回:

-1：失败。

0：成功。

3.2.18 hal_gpio_irq_disable

函数原型:

```
int hal_gpio_irq_disable(uint32_t irq)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

作用：

关闭 GPIO 中断。

参数：

irq：中断号。

返回：

-1：失败。

0：成功。

3.2.19 hal_gpio_init

函数原型：

```
int hal_gpio_init(void)
```

作用：

GPIO 驱动初始化。

参数：

无。

返回：

-1：失败。

0：成功。

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 4 章 功能开发

4.1 功能概述

GPIO 驱动提供了以下核心功能：

- 引脚配置：支持将 GPIO 引脚配置为输入、输出或特殊功能模式。
- 读取引脚状态：读取 GPIO 引脚的当前状态（高电平或低电平）。
- 设置引脚状态：设置 GPIO 引脚的状态（高电平或低电平）。
- 多路复用：支持将 GPIO 引脚配置为多种功能，如 I2C、SPI、UART 等通信接口。
- 中断配置：配置 GPIO 引脚为中断输入，并设置中断触发条件（上升沿、下降沿、双边沿）。

4.2 开发流程

4.2.1 配置 GPIO 为输入模式并读取状态

步骤 1 调用 hal_gpio_pinmux_set_function 设置 GPIO_MUXSEL_IN。

步骤 2 调用 hal_gpio_set_direction 设置 GPIO_DIRECTION_INPUT。

步骤 3 调用 hal_gpio_set_pull 设置内部上下拉。

步骤 4 调用 hal_gpio_get_data 获取输入电平状态。

说明

步骤 1 到步骤 3 没有严格顺序要求

4.2.2 配置 GPIO 为输出模式并设置输出电平

步骤 1 调用 hal_gpio_pinmux_set_function 设置 GPIO_MUXSEL_OUT。

步骤 2 调用 hal_gpio_set_direction 设置 GPIO_DIRECTION_OUTPUT。

步骤 3 调用 hal_gpio_set_pull 设置内部上下拉。

步骤 4 调用 hal_gpio_set_driving_level 设置驱动能力。

步骤 5 调用 hal_gpio_set_data 设置 GPIO_DATA_LOW 或 GPIO_DATA_HIGH。

说明

步骤 1 到步骤 4 没有严格顺序要求

4.2.3 配置 GPIO 为其它模块功能引脚

步骤 1 调用 hal_gpio_pinmux_set_function 设置具体的 GPIO_MUXSEL_FUNCTIONX。

步骤 2 调用 hal_gpio_set_pull 设置内部上下拉。

步骤 3 调用 hal_gpio_set_driving_level 设置驱动能力。

说明

步骤 1 到步骤 3 没有严格顺序要求

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

4.2.4 配置 GPIO 中断模式

步骤 1 调用 hal_gpio_pinmux_set_function 设置为中断功能。

步骤 2 调用 hal_gpio_set_direction 设置 GPIO_DIRECTION_INPUT。

步骤 3 调用 hal_gpio_to_irq 获取相应 GPIO 的 IRQ 中断号。

技巧

需通过 hal_gpio_to_irq 确认 IRQ 号未被占用, 避免中断冲突导致功能异常。

步骤 4 调用 hal_gpio_irq_request 申请中断和注册中断处理函数。

步骤 5 调用 hal_gpio_irq_enable 使能 GPIO 中断。

说明

步骤 1 到步骤 3 没有严格顺序要求

4.3 编程示例

1. 测试 demo 路径

lichee/rtos-hal/hal/test/gpio/test_gpio.c

2. 驱动调用示例

GPIO 一般在 sys_config 中配置，因此驱动需要先从 sysconfig 中获取 GPIO 信息。

如 UART 驱动所示:

```
static void uart_pinctrl_init(uart_port_t uart_port)
{
#ifdef CONFIG_DRIVER_SYSCONFIG
    user_gpio_set_t gpio_cfg[4];
    int count, i;
    char uart_name[16];
    gpio_pin_t uart_pin[4];
    gpio_muxsel_t uart_muxsel[4];

    memset(gpio_cfg, 0, sizeof(gpio_cfg));
    sprintf(uart_name, "uart%d", uart_port);
    count = hal_cfg_get_gpiosec_keycount(uart_name);
    if (!count)
    {
        UART_ERR("[uart%d] not support in sys_config\n", uart_port);
        return ;
    }
    hal_cfg_get_gpiosec_data(uart_name, gpio_cfg, count);

    for (i = 0; i < count; i++)
    {
        uart_pin[i] = (gpio_cfg[i].port - 1) * 32 + gpio_cfg[i].port_num;
        uart_muxsel[i] = gpio_cfg[i].mul_sel;

        hal_gpio_pinmux_set_function(uart_pin[i], uart_muxsel[i]);
        hal_gpio_set_pull(uart_pin[i], GPIO_PULL_UP);
    }
#endif
}
```

1. hal_cfg_get_gpiosec_keycount 获取 uartx 节点 GPIO 配置数目。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

2. hal_cfg_get_gpiosec_data 获取所有 GPIO 配置信息保存到 gpio_cfg 数组。
3. 遍历 gpio_cfg 数组保存的配置并逐一调用 GPIO 驱动接口设置。

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 5 章 调试方法

5.1 调试信息

GPIO 驱动默认不输出 DEBUG 信息，若需要确认 API 调用情况，需要配置 CONFIG_DRIVERS_GPIO_DEBUG。

路径：lichee/rtos-hal/include/hal/hal_gpio.h

```
#define CONFIG_DRIVERS_GPIO_DEBUG
#ifdef  CONFIG_DRIVERS_GPIO_DEBUG
#define GPIO_INFO(fmt, arg...) hal_log_info(fmt, ##arg)
#else
#define GPIO_INFO(fmt, arg...) do {}while(0)
#endif
```

5.2 调试工具

- 配置 CONFIG_HAL_TEST_GPIO 后，可以使用 hal_gpio/hal_gpio_output/hal_gpio_input/hal_gpio_irq 等命令。
- 配置 CONFIG_DRIVERS_GPIO_RECORD_USAGE 后，可以使用 hal_gpio_info 命令。

5.2.1 hal_gpio

- 使用方法：hal_gpio
- 用途：基础 API 测试 Demo

使用示例：

```
cpu0>hal_gpio
Original: pin: 1 pull state: 0, dir: 15, data: 0x0, function_index: 15
Setting: pin: 1 pull state: 1, dir: 1, data: 0x1, function_index: 1
Results: pin: 1 pull state: 1, dir: 1, data: 0x1, function_index: 1
Test hal_gpio_set_pull API success!
Test hal_gpio_set_direction API success!
Test hal_gpio_set_data API success!
Test hal_gpio_pinmux_set_function API success!
Test hal_gpio_get_pull API success!
Test hal_gpio_get_direction API success!
Test hal_gpio_get_data API success!
Test hal_gpio_pinmux_get_function API success!
Test hal_gpio_to_irq API success!
Test hal_gpio_irq_request API success!
Test hal_gpio_irq_enable API success!
Test hal_gpio_irq_disable API success!
Test hal_gpio_irq_free API success!
Test gpio hal APIs success!
```

5.2.2 hal_gpio_output

- 使用方法：hal_gpio_output PXX value
- 用途：GPIO 输出控制测试

5.2.3 hal_gpio_input

- 使用方法：hal_gpio_input PXX
- 用途：GPIO 输入状态读取测试

使用示例：

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
uart>hal_gpio_intput PA3
Will read PA3 status for 10 seconds
Read gpio data test, [PA3]: 1
Read gpio data test, [PA3]: 1
Read gpio data test, [PA3]: 1
```

5.2.4 hal_gpio_irq

- 使用方法：hal_gpio_irq enable PXX
- 用途：GPIO 中断测试

使用示例：

```
uart>hal_gpio_irq enable PA03
pin: 3
gpio 3 to irq 203 succeed!
Test hal_gpio_to_irq API success!
[hal_gpio_irq_request]gpio_desc address is 0x42000000.
request irq 203 succeed!
Test hal_gpio_irq_request API success!
[hal_gpio_irq_enable]gpio_desc address is 0x42000000.
Test hal_gpio_irq_enable API success!
```

5.2.5 hal_gpio_info

- 使用方法：hal_gpio_info [PXX]
- 用途：查看 GPIO 配置信息和使用情况

使用示例 1：查看所有 GPIO 注册使用情况

```
uart>hal_gpio_info
====== GPIO information in use ======
[PC04]: mul_sel:4, pull:1, drv:0, irq:0
[PC05]: mul_sel:4, pull:1, drv:0, irq:0
uart>
```

使用示例 2：查看指定 GPIO 配置 (用于排查电平异常)

```
uart>hal_gpio_info PA3
[PA03]: mul_sel:0, pull:1, drv:2, data:1, dir:1, irq:0
```

技巧

若 PA3 电平异常, 可通过 hal_gpio_info PA3 查看当前配置, 确认复用功能 (mul_sel)、上下拉 (pull)、驱动能力 (drv) 等参数是否正确。

说明

在 V821 平台上才开始添加，旧平台上不一定支持

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 6 章 FAQ

6.1 常见问题及解决方法

6.1.1 GPIO 中断无法触发

问题现象:

配置 GPIO 中断后, 外部信号变化时中断处理函数未被调用。

可能原因:

1. IRQ 中断号已被其他模块占用
2. 中断触发模式配置错误
3. 多核平台中断分配不正确
4. 中断未正确使能

解决步骤:

1. 使用 hal_gpio_to_irq 获取 IRQ 号, 确认未被占用
2. 检查 gpio_interrupt_mode_t 配置是否与实际信号匹配 (上升沿/下降沿/双边沿/高电平/低电平)
3. 确认已调用 hal_gpio_irq_enable 使能中断
4. 使用 hal_gpio_info 命令查看 irq 字段是否为 1

6.1.2 GPIO 电平读取值异常

问题现象:

调用 hal_gpio_get_data 读取的电平值与实际硬件测量值不符。

可能原因:

1. GPIO 未配置为输入模式
2. 上下拉配置不当导致悬空引脚读取不稳定
3. 复用功能设置错误, 引脚被其他外设占用
4. 外部电路驱动能力不足

解决步骤:

1. 确认已调用 hal_gpio_set_direction 设置为 GPIO_DIRECTION_INPUT
2. 确认已调用 hal_gpio_pinmux_set_function 设置为 GPIO_MUXSEL_IN
3. 使用 hal_gpio_info PXX 查看当前配置, 检查 mul_sel 是否为 0(GPIO 功能)
4. 根据硬件电路配置合适的上下拉状态
5. 使用示波器或万用表测量引脚实际电压, 排除硬件问题

6.1.3 GPIO 输出电平无法改变

问题现象:

调用 hal_gpio_set_data 设置高低电平, 但万用表测量引脚电压无变化。

可能原因:

1. GPIO 未配置为输出模式
2. 驱动能力设置过低, 无法驱动外部负载
3. 引脚被其他外设复用占用

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

4. 外部电路短路或过载

解决步骤:

1. 确认已调用 hal_gpio_set_direction 设置为 GPIO_DIRECTION_OUTPUT
2. 确认已调用 hal_gpio_pinmux_set_function 设置为 GPIO_MUXSEL_OUT
3. 使用 hal_gpio_set_driving_level 提高驱动能力 (尝试 GPIO_DRIVING_LEVEL2 或 LEVEL3)
4. 使用 hal_gpio_info PXX 查看配置, 确认 dir:1(输出) 且 mul_sel:1(输出功能)
5. 断开外部电路, 测试 GPIO 空载时能否正常输出

6.1.4 系统休眠唤醒后 GPIO 配置丢失

问题现象:

系统从待机或休眠状态唤醒后,GPIO 功能异常, 需要重新配置。

可能原因:

1. 休眠时 GPIO 电源域被关闭
2. 部分平台唤醒后寄存器配置丢失

解决步骤:

1. 检查确认 GPIO 是否属于常供电域
2. 检查休眠唤醒回调函数中重新初始化 GPIO 的配置
3. 对于需要保持状态的引脚，使用支持状态保持的 GPIO 组 (如 GPIOL)

6.2 一号通 FAQ 列表

更多问题请访问全志一号通平台搜索 “GPIO” 关键词。

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

著作权声明

版权所有 ©2026 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。